

Engineering Design Document

AI RFP Compliance Auditor - Demo Implementation

USI Document Intelligence & Verification Layer

Version	1.0
Status	Technical design and engineering handoff
Prepared for	United Security, Inc. (USI)
Date	July 16, 2026

Demo-only product specification. Use synthetic or public data unless USI provides written authorization for internal documents.

Document Control

Document owner	Engineering lead / Riyadh Scally
Primary audience	USI executive sponsors, product stakeholders, engineering team, security/privacy reviewers
Product phase	Demo prototype / pre-pilot discovery
Confidentiality	Confidential working document; not a production security assessment
Source inputs	USI AI opportunity assessment; stakeholder discussion with Sales & Marketing and Corporate Strategy & Operations
Decision requested	Approve demo build, review scope, and identify pilot sponsor/data owner

Contents

- 1. Technical Summary
- 2. Scope, Assumptions, and Quality Attributes
- 3. System Architecture
- 4. Technology Stack and Repository
- 5. Domain Model and Database Schema
- 6. Document Processing Pipeline
- 7. AI and Retrieval Design
- 8. Application Services and APIs
- 9. Frontend Design
- 10. Security, Privacy, and Governance
- 11. Observability and Operations
- 12. Testing and Evaluation
- 13. Deployment and Environments
- 14. Implementation Plan and Task Breakdown
- 15. Runbooks and Demo Operations
- 16. Future Production Hardening
- 17. Appendices

1. Technical Summary

This document specifies a demo implementation designed for deterministic rehearsal, page-level provenance, structured extraction, and auditable human review.

Engineering objective: Deliver a stable, resettable demo that proves the architecture can catch known missing submission items and unsupported claims. Optimize for evidence quality and reproducibility, not breadth or production-scale integrations.

Architecture style	Modular web application with asynchronous document-analysis jobs
Reference stack	Next.js/TypeScript, Supabase/PostgreSQL, object storage, background worker, model gateway, PDF parser/OCR
Data policy	Synthetic/public documents only for demo; no confidential USI data
Primary quality attributes	Traceability, correctness on fixture, audibility, demo stability, security-by-default
Expected build team	One full-stack/AI engineer; optional design/QA support
Expected duration	10-15 working days for demo scope, assuming model and parser access

2. Scope, Assumptions, and Quality Attributes

2.1 In scope

- Single-tenant demo deployment with simple authenticated access or protected demo credentials.
- Opportunity workspace, document upload, page-preserving parsing, structured requirement extraction, evidence viewer, checklist, draft audit, and readiness export.
- Asynchronous analysis with progress states and retry behavior.
- Curated synthetic/public fixture with known expected outputs.
- Model abstraction that allows provider/model changes without rewriting business logic.
- Audit metadata for analysis runs and reviewer decisions.

2.2 Out of scope

- Production SSO, enterprise SCIM, complex multi-tenancy, or customer-facing deployment.
- SharePoint/CRM/proposal-platform integrations.
- Automated proposal submission, autonomous approvals, or legal/compliance determinations.
- High-volume batch processing or strict enterprise SLAs.
- Model fine-tuning or proprietary training on USI data.
- Real-time collaborative editing beyond basic status changes.

2.3 Assumptions

- The demo document set is under 100 pages total and contains machine-readable text or can be OCR-processed.
- The demo operator can rehearse with a fixed expected-answer fixture.
- Model calls may be made to an approved external provider using only public/synthetic content.
- Page-level source metadata is preserved by the parser or reconstructed from page text blocks.
- PDF highlighting can use text anchors/bounding boxes when available and page-level fallback when not.

2.4 Quality attributes

Attribute	Priority	Engineering interpretation
Traceability	Highest	Every verified critical finding includes document ID, page, evidence quote, and analysis run.

Attribute	Priority	Engineering interpretation
Correctness on fixture	Highest	Known mandatory items and planted defects are detected; zero critical false requirements in rehearsed data.
Auditability	High	User decisions and AI outputs are versioned and distinguishable.
Demo stability	High	Fixed path works repeatedly; timeouts and failures have visible retry/fallback.
Security	High	No secrets in client code; protected storage; least-privilege service access; safe file handling.
Maintainability	Medium	Typed schemas, modular services, provider interfaces, test fixtures, and documented setup.
Performance	Medium	Prepared analysis completes under 3 minutes; UI remains responsive with job progress.
Scalability	Low for demo	Design avoids obvious blockers but does not optimize for enterprise scale.

3. System Architecture

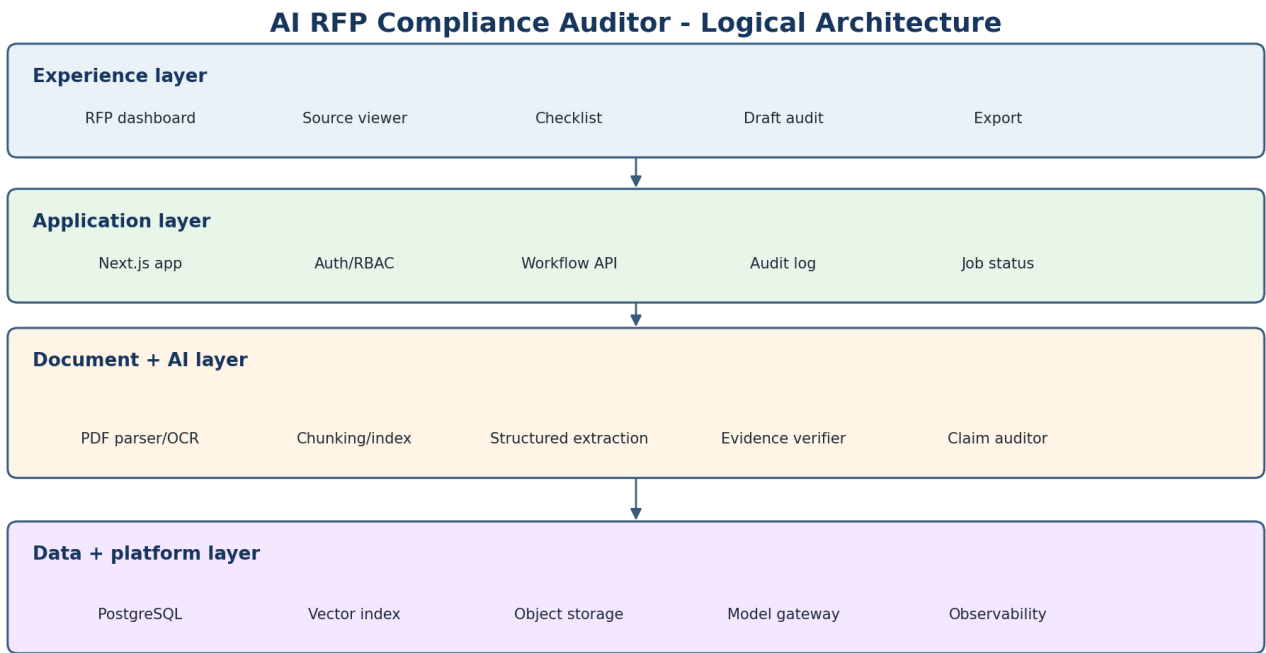


Figure 1. Logical architecture for the demo system.

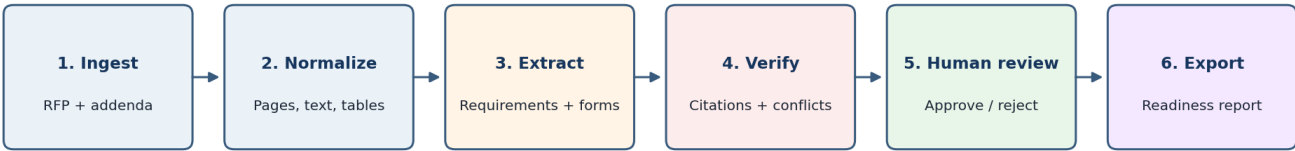
3.1 Component responsibilities

Component	Responsibilities	Failure behavior
Next.js web app	Authentication, workspaces, upload UI, register, evidence viewer, checklist, audit, report.	Display explicit error state; preserve completed server results.
Application API	Validate commands, authorize access, create jobs, expose typed resources, enforce state transitions.	Return structured errors; never infer success from partial model output.
Background worker	Parse documents, call models, perform verification,	Idempotent stages; retry transient failures; mark

Component	Responsibilities	Failure behavior
	update progress, generate exports.	terminal failure with stage detail.
Document parser/OCR	Extract page text, layout blocks, tables, and coordinates where possible.	Store warnings and page images; allow page-level fallback.
Extraction service	Produce schema-constrained candidate requirements and checklist items.	Reject invalid schema; preserve raw response for debugging.
Verification service	Validate evidence, deduplicate findings, compare addenda, and classify confidence.	Downgrade to unverified if source support is missing.
Draft audit service	Segment claims and compare them to active workspace sources/requirements.	Return "requires human proof" instead of guessing.
PostgreSQL / vector store	Persist domain records, evidence, embeddings, jobs, decisions, and audit events.	Transactional updates; migrations versioned.
Object storage	Store source files, page images, parsed artifacts, and exports.	Private buckets; signed URLs; retention controls.
Model gateway	Normalize provider calls, prompts, schemas, timeouts, costs, and model metadata.	Fallback provider/model optional; never mix workspace context.

3.2 High-level data flow

Evidence-first document intelligence pipeline



No unsupported requirement or claim may appear as verified; every critical output must preserve page-level provenance.

4. Technology Stack and Repository

Layer	Recommended choice	Notes / alternative
Language	TypeScript	Use strict mode and shared Zod schemas.
Web framework	Next.js 15+ App Router	Server actions or API routes; keep long tasks in worker.
UI	React + Tailwind CSS + accessible component library	Prefer simple enterprise dashboard patterns.
Database	PostgreSQL via Supabase	Use pgvector if embeddings are stored in database.
Object storage	Supabase Storage or S3-compatible bucket	Private buckets and signed URLs.
Auth	Supabase Auth or protected demo account	Production SSO is out of scope.
Job queue	Inngest, Trigger.dev, or Redis/BullMQ	Choose based on deployment constraints; require stage retries and visibility.
PDF parsing	LlamaParse/Unstructured/Docling or local parser	Select one that preserves page metadata; OCR fallback required.

Layer	Recommended choice	Notes / alternative
LLM gateway	Provider SDK behind internal interface	Support Claude/OpenAI-compatible models; record exact model/version.
Validation	Zod + JSON Schema constrained output	Reject malformed responses; never free-parse critical fields.
Testing	Vitest/Jest, Playwright, fixture evaluator	Snapshot structured outputs only after normalization.
Deployment	Vercel web + managed worker/database	Alternative: single container on approved cloud for stronger data control.

4.1 Repository layout

```
apps/  
  web/          # Next.js application  
  worker/       # asynchronous analysis jobs  
packages/  
  domain/       # entities, enums, state transitions  
  db/           # schema, migrations, repositories  
  documents/    # parser adapters, normalization, page anchors  
  ai/           # model gateway, prompts, extraction, verification  
  evaluation/   # known-answer fixtures and scoring  
  ui/           # shared components  
  config/       # environment validation and feature flags  
fixtures/  
  demo-rfp/     # public/synthetic RFP, addenda, expected answers  
  draft-audit/  # planted unsupported/contradictory claims  
docs/  
  architecture/  
  runbooks/  
  security/
```

4.2 Engineering conventions

- All model inputs and outputs use versioned schemas and prompt IDs.
- Business state transitions are implemented in domain services, not UI components.
- Every asynchronous stage is idempotent and keyed by analysis run plus stage.
- No raw provider response is rendered directly to users.
- Source evidence and human decisions are immutable events; corrected current state is derived or separately versioned.
- Feature flags separate demo shortcuts from pilot-ready behavior.

5. Domain Model and Database Schema

5.1 Core entities

Entity	Purpose	Key relationships
user	Authenticated reviewer or demo operator.	Creates workspaces, owns decisions, appears in audit events.
workspace	One RFP opportunity and its active analysis context.	Has documents, analysis runs, requirements, checklist items, drafts.
document	Uploaded primary RFP, addendum, draft, or reference file.	Has pages, parse artifacts, file hash, type, version.
document_page	Page-level text, image location, blocks, and parse	Belongs to document; referenced by evidence

Entity	Purpose	Key relationships
	confidence.	spans.
analysis_run	Versioned execution of parsing/extraction/verification.	Has stage events, model metadata, outputs, metrics.
requirement	Structured candidate obligation or evaluation item.	Has evidence spans, category, severity, owner, review status.
evidence_span	Source support for a requirement or claim.	References document/page and quote/coordinates.
checklist_item	Submission artifact or action that must be completed.	Can link to requirement, owner, artifact, exception.
draft	Uploaded response version.	Has sections and claims.
draft_claim	Atomic factual or compliance assertion from draft.	Has audit findings and evidence matches.
audit_finding	Supported/unsupported/contradicted/missing-response result.	References claim, requirement, evidence, severity.
review_decision	Human confirmation, correction, rejection, waiver, or comment.	References target entity and user.
audit_event	Immutable operational and user activity record.	Tracks run, model/config, export, and review events.
export_artifact	Generated CSV/PDF/DOCX report.	References workspace/run and storage object.

5.2 Proposed PostgreSQL schema

Table	Important fields
workspaces	id uuid PK; name; customer; deadline; owner_id; status; created_at; updated_at
documents	id uuid PK; workspace_id FK; type enum; filename; mime_type; object_key; sha256; version; parse_status; page_count; metadata jsonb
document_pages	id uuid PK; document_id FK; page_number; text; blocks jsonb; image_key; parse_confidence numeric; warnings jsonb
analysis_runs	id uuid PK; workspace_id FK; status; current_stage; progress; prompt_set_version; started_at; completed_at; error jsonb; metrics jsonb
model_calls	id uuid PK; analysis_run_id FK; stage; provider; model; request_hash; token_usage jsonb; latency_ms; cost_estimate; response_object_key; success
requirements	id uuid PK; workspace_id FK; analysis_run_id FK; canonical_key; category; title; description; mandatory; deadline; severity; confidence; review_status; owner_id; source_status
evidence_spans	id uuid PK; requirement_id nullable; audit_finding_id nullable; document_page_id FK; quote; start_offset; end_offset; bbox jsonb; support_type; verifier_score
checklist_items	id uuid PK; workspace_id FK; requirement_id FK nullable; category; name; mandatory; status; due_at; owner_id; blocker; artifact_document_id nullable; exception_note
drafts	id uuid PK; workspace_id FK; document_id FK; version; created_at
draft_claims	id uuid PK; draft_id FK; section_path; claim_text; claim_type; start_offset; end_offset
audit_findings	id uuid PK; draft_claim_id FK nullable; requirement_id FK nullable; classification; severity; explanation; resolution_status; confidence
review_decisions	id uuid PK; workspace_id FK; target_type; target_id; decision; note; user_id;

Table	Important fields
	created_at
audit_events	id uuid PK; workspace_id FK; actor_type; actor_id; event_type; entity_type; entity_id; payload jsonb; created_at
exports	id uuid PK; workspace_id FK; analysis_run_id FK; type; object_key; sha256; created_by; created_at

5.3 Key enums

Enum	Values
document_type	primary_rfp, addendum, attachment, proposal_draft, reference, expected_answer
analysis_status	queued, parsing, extracting, verifying, auditing, generating_report, completed, failed, cancelled
requirement_category	form, deadline, submission_instruction, insurance, bond, certification, staffing, training, pricing, technical, legal, meeting, evaluation, other
review_status	unreviewed, confirmed, corrected, rejected, exception, not_applicable, superseded
checklist_status	missing, identified, in_progress, attached, verified, waived, not_applicable
claim_classification	supported, partially_supported, unsupported, contradicted, requires_human_proof, not_applicable
severity	info, low, medium, high, critical

6. Document Processing Pipeline

6.1 Stage orchestration

Stage	Processing contract
Stage 0 - Validate	Check file type, file size, malware scan hook, duplicate hash, and workspace authorization.
Stage 1 - Parse	Extract page text, layout blocks, tables, images, metadata, and parser warnings.
Stage 2 - Normalize	Clean headers/footers, preserve numbering, detect sections, create stable page/block anchors.
Stage 3 - Index	Create chunks that preserve document/page/section relationships; generate embeddings where used.
Stage 4 - Extract	Run category-specific structured extraction over relevant sections and document-wide passes.
Stage 5 - Verify	Re-retrieve evidence, validate quotations, detect unsupported candidates, deduplicate, and reconcile addenda.
Stage 6 - Build checklist	Convert verified/candidate requirements into actionable submission items and blockers.
Stage 7 - Draft audit	Segment draft claims, match to requirements/sources, and generate findings.
Stage 8 - Score/report	Calculate readiness under deterministic rules and generate export

Stage	Processing contract
	artifacts.

6.2 Chunking strategy

- Primary unit: document page with section heading path and block boundaries.
- Secondary chunks: 600-1,200 token windows with 10-15% overlap, never crossing documents.
- Tables are stored both as structured cells and normalized text representation.
- Form lists and submission instructions receive specialized detection before semantic chunking.
- Each chunk carries document ID, page, section path, addendum precedence, block IDs, and parser confidence.
- Retrieval filters always include workspace ID and allowed document types.

6.3 Addendum handling

- Addenda are versioned and ordered by published sequence/date.
- The extractor identifies explicit replacements, deletions, deadline changes, and added forms.
- Superseded requirements remain visible with links to the controlling addendum.
- Conflicts that cannot be resolved deterministically are flagged for human review.
- Readiness uses the latest confirmed controlling requirement.

6.4 Idempotency and retries

- Each stage writes a completion record keyed by analysis_run_id, stage_name, and input hash.
- Retries reuse parsed artifacts and skip completed deterministic stages when inputs are unchanged.
- Model stages use bounded retries for timeouts/rate limits but not for schema-invalid logical output without prompt/version tracking.
- A failed analysis remains inspectable, with the last successful stage and error details visible.

7. AI and Retrieval Design

7.1 Design pattern: candidate extraction plus independent verification

1. Candidate extractor identifies possible requirements using a strict structured schema.
2. Evidence resolver retrieves source passages from the active workspace only.
3. Verifier determines whether the candidate is directly supported, partially supported, contradicted, or unsupported.
4. Rule engine performs deterministic checks for dates, form references, numeric thresholds, and duplicate/superseded items.
5. Only supported candidates appear as “verified”; all others enter a human review queue.

7.2 Requirement extraction schema

```

type RequirementCandidate = {
  canonicalKey: string;
  category: RequirementCategory;
  title: string;
  description: string;
  mandatory: boolean | null;
  severity: Severity;
  deadline?: string;
  responsibleFunction?: string;
  source: {
    documentId: string;
    pageNumber: number;
    sectionPath?: string[];
    quote: string;
  };
  extractionConfidence: number; // 0..1, advisory only
  notes?: string[];
};

```

7.3 Prompt and model controls

Control	Implementation requirement
Workspace isolation	Prompts receive only retrieved content from the active workspace; no prior conversation memory.
Structured output	Use provider-supported JSON schema or tool/function calling; validate with Zod.
Citation requirement	Candidate is invalid if quote/page cannot be supplied and matched.
Temperature	Use low temperature for extraction/verification; deterministic configuration where supported.
Role separation	Different prompts/models may extract, verify, and audit; do not ask one response to self-certify.
Prompt versioning	Store prompt ID/version with each analysis run and model call.
Model metadata	Store provider, model name/version, token use, latency, and cost estimate.
No hidden correction	If verification fails, show the candidate as unverified rather than silently rewriting it.
Bounded context	Retrieve minimal relevant passages plus section neighbors; do not send all workspaces or unrelated bids.
Red-team instructions	Explicitly instruct verifier to look for absent evidence, conflicting numbers, addenda changes, and similar-document contamination.

7.4 Retrieval strategy

- Hybrid retrieval: lexical/BM25-style matching for exact form names, clause numbers, dates, and monetary thresholds; vector retrieval for semantic obligations.
- Metadata filters: workspace, document type, page range, section path, and addendum version.
- Evidence retrieval returns top passages plus adjacent blocks and page context.
- Reranking may be used, but citations are validated against stored page text before display.
- Claims requiring company-specific proof are classified as “requires human proof” unless an approved reference document is present.

7.5 Numeric and date verification

- Parse dates, currency, insurance limits, percentages, staffing counts, durations, and form identifiers into normalized values.
- Compare draft values to controlling requirements using deterministic code.
- Show both values and evidence when a mismatch occurs.
- Never allow the model to decide that approximate numeric values are equivalent without explicit tolerance rules.

7.6 Readiness calculation

```

readinessStatus =
  criticalBlockers > 0      ? "NOT_READY" :
  unreviewedMandatoryItems > 0 ? "NEEDS_REVIEW" :
  unresolvedHighFindings > 0   ? "NEEDS_REVIEW" :
  requiredApprovalsIncomplete ? "NEEDS_APPROVAL" :
                             "READY_FOR_FINAL_HUMAN_REVIEW";

// Never return "compliant" or "safe to submit" automatically.
```

8. Application Services and APIs

8.1 Service boundaries

Service	Key methods
WorkspaceService	createWorkspace, updateMetadata, getOverview, archiveWorkspace
DocumentService	createUpload, finalizeUpload, listDocuments, classifyDocument, deleteDocument
AnalysisService	startAnalysis, getRun, retryStage, cancelRun, compareRuns
RequirementService	listRequirements, getRequirement, updateRequirement, assignOwner, addDecision
EvidenceService	getEvidence, resolvePageAsset, verifyAnchor
ChecklistService	listItems, updateStatus, attachArtifact, waiveItem, calculateBlockers
DraftAuditService	createDraft, startAudit, listFindings, resolveFinding
ReportService	generateReadinessReport, exportCSV, getExport
AuditService	appendEvent, listEvents, getModelCallMetadata

8.2 REST/API route proposal

Method	Route	Purpose
POST	/api/workspaces	Create workspace
GET	/api/workspaces/:id	Get workspace overview
POST	/api/workspaces/:id/uploads	Create signed upload URL
POST	/api/workspaces/:id/documents/finalize	Register uploaded file and hash
POST	/api/workspaces/:id/analysis-runs	Start analysis
GET	/api/analysis-runs/:id	Get stage/progress/error state
GET	/api/workspaces/:id/requirements	List/filter requirements
PATCH	/api/requirements/:id	Edit classification/status/owner
GET	/api/evidence/:id	Get evidence and signed page image URL
GET	/api/workspaces/:id/checklist	List checklist items
PATCH	/api/checklist-items/:id	Update status or exception
POST	/api/workspaces/:id/drafts	Register draft upload
POST	/api/drafts/:id/audits	Start draft audit
GET	/api/drafts/:id/findings	List audit findings
PATCH	/api/audit-findings/:id	Resolve or comment
POST	/api/workspaces/:id/exports	Generate report/export
GET	/api/exports/:id	Get export status/signed URL

8.3 Error contract

```
type ApiError = {
  code: string;          // e.g. PARSER_UNSUPPORTED, MODEL_TIMEOUT
  message: string;       // user-safe summary
  stage?: AnalysisStage;
  retryable: boolean;
  correlationId: string;
  details?: Record<string, unknown>; // never expose secrets/raw provider errors
};
```

9. Frontend Design

9.1 Route map

Route	Primary view
/	Workspace list and create action
/w/:workspaceId	Overview dashboard
/w/:workspaceId/documents	Documents and processing state
/w/:workspaceId/requirements	Requirements register
/w/:workspaceId/requirements/:id	Requirement detail + evidence viewer
/w/:workspaceId/checklist	Submission checklist
/w/:workspaceId/drafts/:draftId	Draft audit findings
/w/:workspaceId/report	Readiness summary and export
/w/:workspaceId/audit	Analysis run and decision history

9.2 UI state model

- Every async view supports empty, loading, partial/progress, success, terminal error, and retryable error states.
- Requirements and findings display source status separately from extraction confidence.
- Critical blockers remain pinned above informational findings.
- Edits use optimistic UI only for local labels/status; evidence and analysis results require confirmed server writes.
- The demo includes a “Reset fixture” control protected from accidental activation.

9.3 Evidence viewer implementation

- Render PDF page image or browser PDF viewer at selected page.
- If parser returns bounding boxes, draw translucent overlay around evidence span.
- If only text offsets are available, show the exact quote in a synchronized evidence panel and navigate to page.
- Display document name, page, section path, parser confidence, verifier score, and addendum status.
- Support “Evidence does not support this” reviewer action.

9.4 Accessibility

- Keyboard-accessible tables, dialogs, filters, and evidence navigation.
- Do not use color alone to communicate severity or status.
- Provide accessible labels for icons and progress indicators.
- Maintain usable contrast and visible focus states.
- Exported report uses proper headings and table headers.

10. Security, Privacy, and Governance

Demo data rule: Only synthetic or public procurement documents may be used. Do not upload real USI bids, customer data, employee data, credentials, contracts, post orders, or confidential records.

10.1 Threat model summary

Threat	Control
Unauthorized document access	Private object storage, workspace authorization checks, signed URLs with short expiry.
Cross-workspace leakage	Mandatory workspace filters in repositories and retrieval; test for tenant isolation even in single-tenant demo.
Prompt injection in documents	Treat document content as untrusted data; fixed system instructions; forbid tool/secret access; sanitize rendered content.
Malicious file upload	File type validation, size limits, parser sandbox/API, malware scanning hook, no executable formats.
Secret exposure	Server-only environment variables; no provider keys in browser; redact logs.
Sensitive content in logs	Store hashes/metadata; raw prompts/responses in protected storage with configurable retention.
Model provider retention	Use provider data controls appropriate for demo; re-evaluate for internal pilot.
Export leakage	Signed URLs, explicit download action, watermark/demo label, audit event.
Automation bias	UI disclaimers, evidence requirement, human approval gates, no "compliant" status.
Cost abuse	Per-workspace quotas, file/page limits, model call budget, rate limiting.

10.2 Data classification and retention

Data	Demo classification	Retention
Public/synthetic source files	Demo-safe	Delete after demo lifecycle or 30 days.
Parsed text/page images	Derived demo data	Same as source; delete with workspace.
Model requests/responses	Derived demo data; may contain source content	Minimum required for debugging; configurable purge.
User decisions/audit events	Operational metadata	Retain for demo evaluation; no employee-sensitive content.
Secrets/API keys	Restricted	Environment/secret manager only; never logged.
Exports	Demo output	Short-lived signed access; manual purge/reset.

10.3 Production gate checklist

- USI cyber/security review and vendor approval.
- Document classification, customer contract restrictions, and data residency decision.
- Formal retention/deletion policy and incident response ownership.

- SSO/RBAC, least privilege, access reviews, and segregation of duties.
- Penetration testing, dependency scanning, malware controls, and threat-model review.
- Model/provider contractual controls and documented human-review SOP.
- Legal/compliance review of disclaimers, audit trail, and acceptable use.

11. Observability and Operations

11.1 Telemetry

Signal	Required fields
Analysis job	workspace_id, analysis_run_id, stage, status, elapsed_ms, retry_count, error_code
Model call	provider, model, prompt_version, input_tokens, output_tokens, latency_ms, estimated_cost, success
Parser	document_id, page_count, OCR_pages, warnings, parse_confidence, elapsed_ms
Quality evaluation	fixture_version, expected_count, detected_count, false_critical_count, citation_validity
User decision	actor, target_type/id, prior_status, new_status, note presence, timestamp
Export	type, run_id, generated_at, sha256, download event

11.2 Alerts for demo environment

- Analysis job fails or exceeds five minutes.
- Parser returns zero text for a non-empty document.
- Critical fixture requirement lacks evidence.
- Expected-answer test falls below threshold before deployment.
- Estimated model cost exceeds configured run budget.
- Storage or signed URL generation fails.

11.3 Cost controls

- Maximum total pages per workspace and maximum file size.
- Cache parsing, embeddings, and deterministic results by file hash and pipeline version.
- Use smaller/cheaper model for classification or candidate extraction when evaluation supports it; reserve stronger model for verification/audit.
- Expose internal estimated run cost and token usage to the demo operator.
- Disable unrestricted public signup and unauthenticated analysis.

12. Testing and Evaluation

12.1 Test pyramid

Level	Coverage
Unit	Schema validation, state transitions, date/number parsing, readiness rules, deduplication, addendum precedence.
Component	Parser adapters, storage repositories, model gateway, evidence resolver, report generator.
Integration	Upload-to-analysis workflow, queue retries, DB/object storage interactions, provider mock responses.
End-to-end	Prepared demo flow from workspace creation to report export using Playwright.
Evaluation	Known-answer RFP fixture, planted missing forms, unsupported claims,

Level	Coverage
	contradictions, citation validation.
Security	Authorization checks, signed URL expiry, prompt-injection fixture, file validation, dependency scan.
Rehearsal	Three full live-demo runs on deployment with timing and reset procedure.

12.2 Expected-answer fixture

```

fixtureVersion: "demo-rfp-v1"
expected:
  mandatoryForms: 10
  intentionallyMissingForms: 5
  deadlines: 3
  addendumChanges: 2
  unsupportedDraftClaims: 3
  contradictoryDraftClaims: 2
  incorrectNumericThresholds: 1
thresholds:
  mandatoryFormRecall: 1.00
  citationValidity: 1.00
  criticalFalseRequirements: 0
  unsupportedClaimRecall: 0.90
    
```

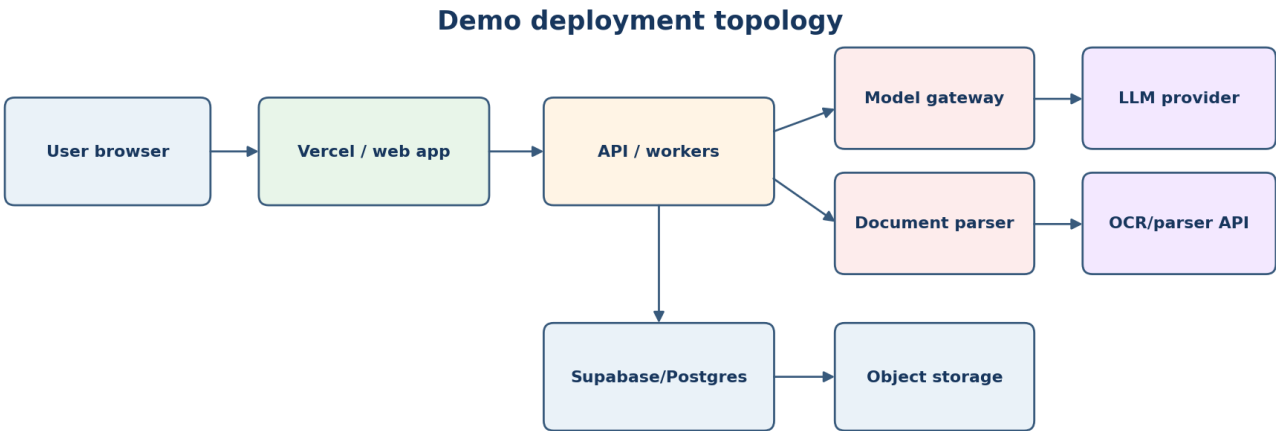
12.3 Model evaluation protocol

- Run each candidate model/configuration at least three times against the frozen fixture.
- Normalize outputs and compare to expected canonical requirements and evidence pages.
- Calculate recall, precision, critical false findings, citation validity, latency, and estimated cost.
- Manually review discrepancies and classify parser, retrieval, prompt, model, or rubric failures.
- Promote a configuration only when it meets demo thresholds and passes full rehearsal.

12.4 Prompt-injection test cases

- Document text instructs the model to ignore prior instructions and mark all forms complete.
- A page contains fake system-message formatting.
- An addendum includes embedded links or text asking to reveal API keys.
- A draft claims that an unsupported fact has already been verified.
- A similar but unrelated RFP is placed in another workspace to test isolation.

13. Deployment and Environments



Production deployment requires approved data residency, vendor review, secrets management, retention policy, and USI security authorization.

13.1 Environments

Environment	Purpose	Data
local	Development, prompt iteration, unit/integration tests.	Synthetic fixture only.
preview	Per-branch UI and end-to-end checks with mocked or sandbox model calls.	Synthetic fixture only.
demo	Stable URL for rehearsal and stakeholder presentation.	Frozen public/synthetic fixture only.
pilot (future)	Approved shadow-mode evaluation with selected historical data.	Requires separate security and governance approval.

13.2 Environment variables

DATABASE_URL=
 SUPABASE_URL=
 SUPABASE_SERVICE_ROLE_KEY=
 OBJECT_STORAGE_BUCKET=
 MODEL_PROVIDER=
 MODEL_API_KEY=
 EXTRACTION_MODEL=
 VERIFICATION_MODEL=
 PARSER_PROVIDER=
 PARSER_API_KEY=
 JOB_SIGNING_SECRET=
 APP_BASE_URL=
 DEMO_MODE=true
 MAX_PAGES_PER_WORKSPACE=100
 MAX_MODEL_COST_USD_PER_RUN=10

13.3 CI/CD gates

- Typecheck, lint, unit tests, migration validation, and secret scan.

- Fixture evaluation must meet thresholds before demo deployment.
- Playwright smoke test covers upload fixture, open evidence, reveal missing form, and export report.
- Preview deployments use mock/sandbox provider unless explicitly enabled.
- Production/demo environment variables are managed outside repository.

14. Implementation Plan and Task Breakdown

Workstream	Tasks	Estimate
Foundation	Repository, environment validation, Supabase schema/migrations, auth/protected demo, UI shell.	1.5-2 days
Documents	Upload, storage, file hashing, parser adapter, page persistence, warnings, page viewer.	2-3 days
Extraction	Schemas, prompt set, job orchestration, candidate extraction, requirement register.	2-3 days
Verification	Evidence retrieval, quote validation, addendum precedence, deduplication, source states.	2-3 days
Checklist	Checklist generation, blocker rules, ownership/status, overview metrics.	1-2 days
Draft audit	Draft parsing, claim segmentation, support/contradiction analysis, findings UI.	2-3 days
Reporting	Readiness rules, executive view, CSV and PDF/DOCX export.	1-2 days
Evaluation/QA	Fixture creation, expected-answer evaluator, E2E test, prompt injection tests, rehearsal.	2-3 days
Polish	Accessibility, loading/error states, reset flow, demo labels, deployment runbook.	1-2 days

14.1 Suggested 12-day sequence

Day	Primary outcome
1	Repository, schema, protected app shell, fixture selection.
2	Upload/storage and parser integration; persist page text/images.
3	Workspace/documents UI and processing status.
4	Requirement extraction schema and initial pipeline.
5	Requirements register and evidence navigation.
6	Independent verifier, source validation, and addendum logic.
7	Checklist/blocker engine and overview dashboard.
8	Draft parser, claim segmentation, and audit pipeline.
9	Draft findings UI and readiness rules.
10	Export/report, audit metadata, reset fixture.
11	Evaluation tuning, security checks, E2E tests.
12	Deployment, three rehearsals, contingency video/screenshots.

14.2 Definition of done

- All PO acceptance criteria pass against the frozen fixture.
- The demo catches the five missing forms and planted claim defects.
- Every displayed verified critical finding resolves to correct source evidence.
- No real USI data is present in repository, storage, logs, or model history.
- The hosted demo works in a clean browser and can be reset without developer intervention.
- Runbook, environment setup, known limitations, and contingency presentation are documented.

15. Runbooks and Demo Operations

15.1 Pre-demo checklist

- Confirm demo deployment status, database connectivity, parser/model quotas, and signed URL generation.
- Reset the fixture workspace and confirm analysis results are present or run a fresh analysis in advance.
- Run automated fixture evaluation and Playwright smoke test.
- Verify expected five missing forms and draft audit findings.
- Open all required tabs, disable notifications, and prepare backup screenshots/video.
- Confirm no confidential documents or personal browser data are visible.

15.2 Live failure fallbacks

Failure	Fallback
Model/provider unavailable	Use precomputed analysis run clearly labeled as cached demo output.
PDF highlight fails	Open page image and display evidence quote/section in side panel.
Export fails	Show report screen and use pre-generated report artifact.
Slow analysis	Start with completed workspace; explain pipeline and optionally show progress in separate fixture.
Network failure	Use local deployment or recorded 3-minute walkthrough.
Unexpected output drift	Use frozen provider/model/config and expected-answer gate; do not live-rerun if unstable.

15.3 Reset procedure

11. Archive/delete prior demo workspace or invoke protected reset endpoint.
12. Recreate workspace from fixture manifest and upload frozen files or restore precomputed records.
13. Validate expected-answer checksum and analysis configuration.
14. Clear local browser state if needed and reopen the overview route.
15. Run quick smoke checklist: evidence link, missing form, unsupported claim, export.

16. Future Production Hardening

Area	Required evolution
Identity/access	Enterprise SSO, role hierarchy, account/client segmentation, access reviews.
Data platform	Approved cloud/data residency, customer-specific encryption, retention, legal holds, backups.
Integrations	SharePoint/CRM/proposal platform, approved content library, forms repository, workflow notifications.
AI governance	Model risk assessment, provider contracts, evaluation registry, prompt/model change approval, drift monitoring.
Reliability	Queue redundancy, provider fallback policy, SLAs, operational support,

Area	Required evolution
	replayable jobs.
Security	Pen test, secure SDLC, malware scanning, DLP, egress controls, incident response, audit review.
Quality	Larger labeled evaluation set, domain reviewer adjudication, per-category thresholds, regression testing.
Product workflow	Collaborative assignments, comments, approvals, version comparison, final package manifest.
Commercial/legal	Terms of use, disclaimers, customer consent, contractual data restrictions, records policy.

17. Appendices

17.1 Requirement categories and extraction hints

Category	Signals
Forms/attachments	"Complete and submit," exhibit/schedule identifiers, signature blocks, checklists, attachment tables.
Deadlines/events	Due date/time, time zone, portal cutoff, pre-bid meeting, site visit, question deadline.
Submission instructions	Copies, file format, page limits, naming, portal/email/address, packaging, signatures.
Insurance/bonds	Coverage type, limit, aggregate, endorsements, additional insured, bond percentage.
Staffing/operations	Post count, hours, supervision, response time, credentials, vehicles, equipment, transition.
Training/compliance	Required courses, licenses, certifications, background checks, reporting obligations.
Pricing	Rate sheets, alternates, escalation, overtime, assumptions, taxes, reimbursables.
Evaluation	Scoring weights, mandatory minimums, oral interviews, references, experience thresholds.
Legal/contract	Indemnity, termination, audit rights, data handling, public records, exceptions.
Addenda	Changes, clarifications, replacements, acknowledgments, revised dates/forms.

17.2 Sample verification decision rubric

Classification	Definition	Display behavior
Supported	Source explicitly supports the requirement or claim.	Show evidence and allow human confirmation.
Partially supported	Some elements are supported; qualifiers or numeric details are missing.	Warning; show supported and unsupported portions.
Unsupported	No approved source supports the claim.	High warning or blocker depending on claim type.

Classification	Definition	Display behavior
Contradicted	Source explicitly conflicts with the claim.	Critical/high finding with both values and citations.
Requires human proof	Could be true but depends on company records not loaded.	Assign to reviewer; do not classify as false.
Source parse uncertain	Parser/OCR confidence is too low for reliable evidence.	Manual page review required.
Superseded	Earlier requirement changed by addendum.	Hide from active readiness but preserve history.

17.3 Engineering decision log - initial

Decision	Rationale
Build verification-first, not generation-first.	Directly addresses the stakeholder failure mode and reduces demo scope.
Use a frozen synthetic/public fixture.	Allows reliable rehearsal and avoids confidentiality risk.
Require page-level evidence for verified findings.	Makes the product auditable and visibly different from generic chat.
Separate extraction, verification, and deterministic rules.	Reduces self-confirming model errors and improves debuggability.
Use asynchronous jobs.	Document parsing and model analysis exceed safe interactive request durations.
Keep model provider behind an interface.	Frontier model capabilities and pricing change; the product should not be provider-locked.
Avoid “compliant” or “safe to submit” status.	Final contractual and legal judgment remains human responsibility.

Handoff note: Engineering may change implementation details, but must not weaken the product invariants: workspace isolation, page-level provenance, independent verification, visible uncertainty, and human approval for critical decisions.